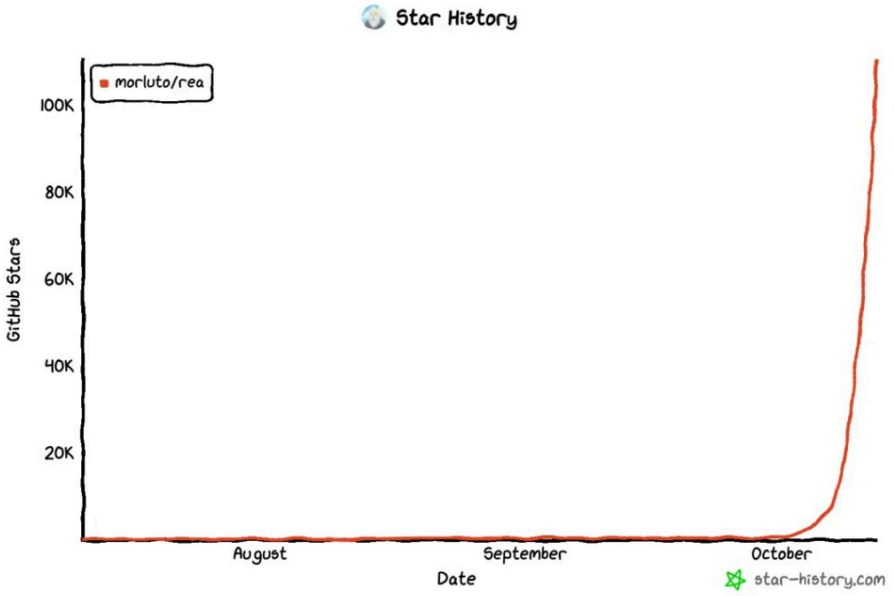
摘要：

这波热潮背后，逆向技术本身并不新鲜，GhidraMCP等工具早已存在，REA的真正突破是将门槛压缩至一行命令，让非专业用户也能拆解任何软件。然而"分析研究"受法律保护，"重建复刻"却无豁免——AI同时读反编译结果又写新代码，直接拆穿了净室开发的法律防火墙。风险最终由用户独自承担。

「在应用里看到一个想加入自己产品的功能？让智能体去帮你调查它。」

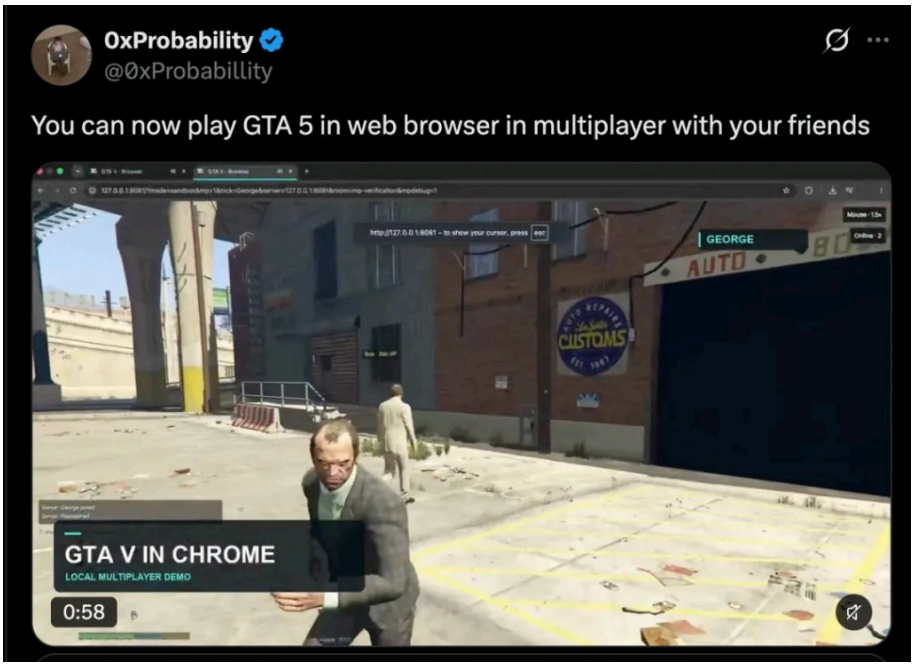
这是 REA 项目主页第一屏的话。后半句更直接：即使没有源代码，智能体也可以检查应用、解释功能怎么工作、展示证据，「并为你的项目实现类似功能」。

REA 全名 Reverse Engineer Anything，即「逆向一切」。**上周它在 AI 开发者圈刷了屏：10 月 6 日还不到 1 万星，到 10 月 12 日已经超过 11 万。**



6 天涨了 10 万星，平均每天约 1.7 万｜图片来源：Star History

真正让普通人看到这件事的，是游戏。10 月初，一批老游戏突然能在浏览器里直接玩了：《光环：战斗进化》《半条命》《暗黑破坏神》，还有真正意义上的 3A 大作《GTA5》和《使命召唤：现代战争 2》。



其中最出圈的一份复盘，来自逆向开发者 Maurice Heumann：他让十几个 Claude Code 和 Codex 智能体并行干活，把一款射击游戏 99% 的函数还原成了 C++ 源码，其中 83% 编译出来和原程序逐字节一致。他用的是 Hex-Rays 的官方 MCP，不是 REA。

也就是说，REA 只是这波浪潮里名字最响的那一个。

同一个星期，整个 AI 逆向圈都在加速。

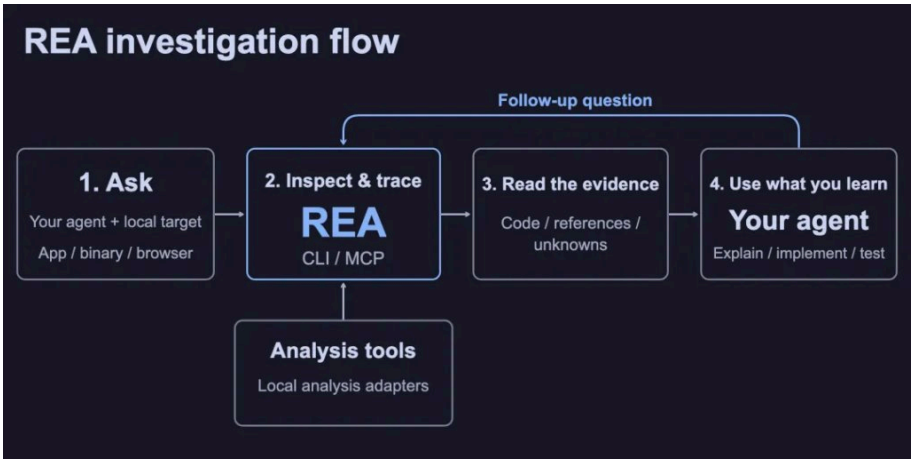
似乎，人人都有免费软件用、免费游戏玩的日子到来了。

01 旧工具，新用户

逆向工程，就是手里没有源代码，从编译好的程序往回推，搞清楚它是怎么写的。这原本是安全研究员和破解者的手艺：在反编译器、反汇编器、调试器之间来回切，一个功能拆上几天是常事。

REA 干的事，是把这套工具接到 AI 编程智能体上。接口用的是 MCP，一种让 Claude Code、Codex、Cursor 这类智能体调用外部工具的通用协议。

装好之后，你对智能体说一句「弄清这个应用的搜索功能怎么工作」，它会自己去调 Ghidra、IDA 或 Hopper 等对应的工具，帮你翻二进制，调 JADX 拆安卓安装包，或者直接读 Electron 应用里打包的 JavaScript，最后把结论和证据一起交回来。



智能体负责提问，REA 负责取证｜图片来源：REA

听上去像个新物种。但放在专业圈里看，它其实更像一次重新包装：毕竟让 AI 接上逆向工具，GhidraMCP、IDA Pro MCP 这些开源项目去年就在做了。

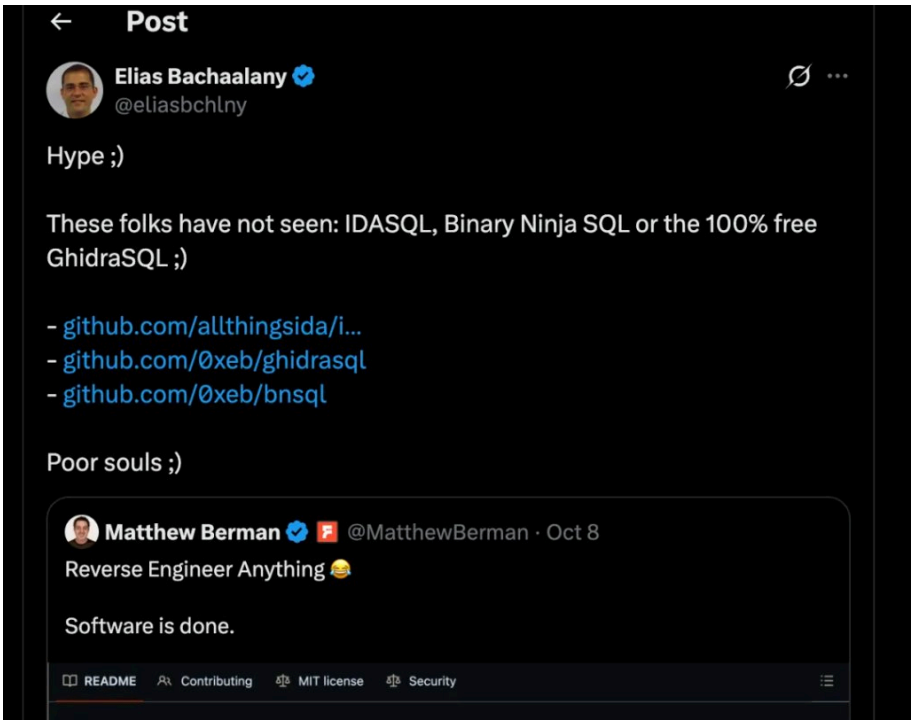
REA 走红前不久，IDA Pro MCP 的作者、调试器 x64dbg 的开发者 Duncan Ogilvie 在 X 上说，他用 IDA 的 MCP 插件，大约一小时解完了今年 Flare-On 的全部题目。



Flare-On 是谷歌旗下 FLARE 团队每年办的逆向挑战赛，题目大多取材自真实的恶意软件分析。

这并非个例，逆向领域的专家们其实普遍对今天这波新兴起的浪潮不是很看得上眼：Elias Bachaalany 是逆向工程教材《Practical Reverse Engineering》的合著者，早年在 IDA 的开发商 Hex-Rays 工作过。

REA 被各路人马转发刷屏时，他也转了其中一条推介，开头就一个词：炒作。接着他列了几个早就存在的同类工具，意思是这些人只是没见过。



逆向教材合著者的评价，开头就一个词｜图片来源：X

AI 能拆程序是真的，只是不从 REA 开始。能拆到什么程度，圈内说法也不一致。

安全领域博主轩辕之风认为，没加壳的程序 AI 轻松拿下，轻度加壳也行，VMP 这类高强度虚拟化保护目前还啃不动；他还批评 REA 连动态调试都没有。一家做逆向产品的公司也在 X 上说，碰到加壳样本，模型分析照样会卡住。反方的说法是：混淆能拖慢 AI，挡不住它。

公开的案例，多数落在容易的那一头。宝玉用 Codex 加 REA 逆向了一个混淆过的网页前端，说几乎完整还原了里面的编辑器。也有博主拿它拆剪映，发现 Claude 拆到一半会卡住，也会被限制。

还有一类东西，它怎么都拿不到：放在服务器上的逻辑。你能拆开装在自己电脑上的客户端，显然拆不开别人机房里的那一半。

另外，工具多也未必是好事。Duncan Ogilvie 说过，他不认为给模型暴露更多工具能让它表现更好；现在也确实已经有 REA 用户抱怨，它塞给智能体的工具说明太长，用不了多久就把上下文挤满了。

REA 真正新的地方，其实在还是在被大幅降低的逆向门槛。

以前想让 AI 帮你逆向，你得先是个会逆向的人：装好 IDA，配好插件，知道该问什么。现在一行 npx rea-agents setup，选好要接入的智能体、点头确认，配置就写好了。把它推上热榜的，也不是安全圈，是 AI 博主和科技大号。

REA 的技术不新，新的是用它的人。

02「逆向」AI 做，「牢」用户坐

新用户带来的第一个问题，就藏在那句宣传语里：给你的项目里加上类似功能。拆开来看，和照着做一个，在法律上是两件事。

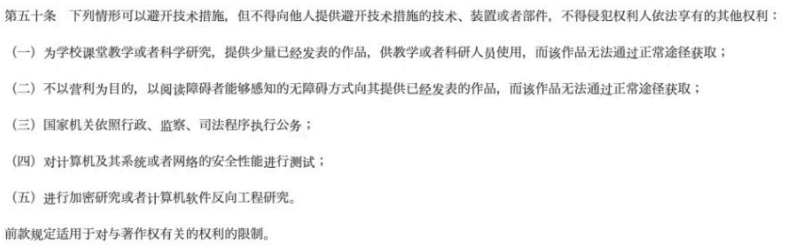
拆一个软件，法律上大致要过三个层面：**商业秘密、版权，还有合同**。

首先是商业机密，最高人民法院 2020 年关于商业秘密案件的司法解释写得很明白：对从公开渠道取得的产品做拆卸、测绘、分析，拿到技术信息，不算侵犯商业秘密。前提是东西来路正，先用不正当手段拿到秘密、再拿逆向当说辞，法院不认。

版权这部分，《计算机软件保护条例》第十七条允许为了学习和研究软件里的设计思想和原理去使用软件，2020 年修正的《著作权法》第五十条，把「计算机软件反向工程研究」列为可以避开技术措施的情形之一。

但大多数商业软件的用户协议里，都有一条禁止反编译的条款。在美国这一条是经常被引用且能执行的：2003 年的 Bowers 诉 Baystate 案里，联邦巡回上诉法院认定，软件许可里的禁止逆向条款有效，逆向之后做出竞品的一方，按违约赔了钱。

欧盟反过来，合同里禁止为互操作而反编译的条款，一律无效。同一件事，换个地方答案就不同。



第五十条　下列情形可以避开技术措施，但不得向他人提供避开技术措施的技术、装置或者部件，不得侵犯权利人依法享有的其他权利：

（一）为学校课堂教学或者科学研究，提供少量已经发表的作品，供教学或者科研人员使用，而该作品无法通过正常途径获取；

（二）不以营利为目的，以阅读障碍者能够感知的无障碍方式向其提供已经发表的作品，而该作品无法通过正常途径获取；

（三）国家机关依照行政、监察、司法程序执行公务；

（四）对计算机及其系统或者网络的安全性能进行测试；

（五）进行加密研究或者计算机软件反向工程研究。

前款规定适用于对与著作权有关的权利的限制。

《著作权法》第五十条：豁免的是「研究」｜图片来源：中国政府网

至于照着做一个，这里其实有个好消息：版权保护的是代码怎么写，不保护功能本身。条例第六条写得清楚，保护不延及开发软件所用的思想、处理过程和操作方法。单从版权看，别人有个好用的搜索功能，你自己写一个效果相似的，本身不违法。

麻烦其实出在它是怎么写出来的。软件行业应对这件事有一套老办法，叫**净室开发：一组人去拆对方的产品，只写出功能说明书；另一组从没看过对方代码的人，照着说明书从头写。**两组之间隔着一道墙，墙就是日后打官司时的证据，证明新代码是独立写出来的。

REA 现在把这道墙拆了：读反编译结果的是这个智能体，写新代码的还是这个智能体，中间没有任何隔断。Hacker News 上有人把这种做法叫作「洗知识产权」；一个复刻 Photoshop 的项目，开发者说他们团队明确禁止成员用 Ghidra 和 REA，原因很简单，怕被起诉。

游戏圈把这件事推到了极端。Maurice 那种做法叫字节级匹配反编译：写出来的源码，编译之后要和原程序一个字节都不差，他说这给了智能体一个非对即错的客观信号。净室追求的是「我没照着你写」，字节级匹配追求的恰恰是「写得和你一模一样」。

发行商的反应也很直接。**10 月 6 日，一个能在浏览器里玩 GTA 5 的网站上线，几个小时后就被下架，不过这一例背后还有 2023 年泄露的源码，未必全靠逆向。据 Destructoid 的盘点，Rockstar、Activision、Bethesda 法务都已经光速触动，撤下了自家游戏的浏览器版本。**

回头再看 REA 的免责声明，它说自己为「合法的逆向工程研究、分析和重建」提供工具。研究和分析，上面这些法条都留了门；重建这个词，找不到对应的豁免。项目把它写进了合法用途，又把「取得授权、遵守法律」的责任交给了你。



「重建」被写进了合法用途｜图片来源：REA

还有一层风险在数据上。REA 的说明里写着分析在本机运行，这话没错，工具确实跑在你自己电脑上。但智能体要理解这些分析结果，就得把反编译出来的代码片段、调用关系和字符串，发给背后的模型服务商。如果你拆的是客户交给你的软件，或者签过保密协议的东西，这一步本身就可能违约。

模型服务商也有自己的底线。前面那位拆剪映的博主遇到的卡顿和限制，就是这条底线在起作用。它算是一道护栏，只是拦在哪里，由服务商说了算。

数一数这里面的几方：项目说，责任在你；模型会拦你，但标准不透明；被拆的软件公司，手里握着用户协议和律师。真正站在风险正中间的，只有发出那句指令的用户。

至于《著作权法》第五十条，它在列出豁免之后，紧跟了一句限制：不得向他人提供避开技术措施的技术、装置或者部件。一个人在自己电脑上做研究，和把这种能力打包成一行命令发给十几万人，是不是一回事，法条没有给答案。看起来，这个问题迟早也会被问到 REA 这类项目本身头上，而不只是它的用户。

这几天的公开讨论里，职业逆向工程师谈能力、谈工具、谈强壳，很少有人谈合规。这不奇怪：拿到授权再动手，发现漏洞先通知厂商，为了兼容去拆协议，这些是这一行默认的规矩，不需要每次重新说。但 REA 把这门手艺打包成了一行命令，这些规矩显然没法跟着一起打包。

对做 App 的人来说，这也是一次提醒。Elias Bachaalany 的建议很直接：只要程序跑在用户手里，就能被拆开，唯一稳妥的办法是把关键逻辑放到服务器上。他在后面补了一句：目前是这样。

而对用 REA 的用户来说，在让智能体动手之前，或许值得先把 REA 的主页拉到最底下，读一遍那段免责声明：拿过来之后的事，它其实早就说好了，要你自己来担。

本文来源：极客公园

风险提示及免责条款

市场有风险，投资需谨慎。本文不构成个人投资建议，也未考虑到个别用户特殊的投资目标、财务状况或需要。用户应考虑本文中的任何意见、观点或结论是否符合其特定状况。据此投资，责任自负。



限免

32 收藏 分享到:

写评论

请发表您的评论

表情 图片

发布评论